

T2 · Marco 4

Flight Routes Check

CSES 1682 — conectividade forte com duas DFS

Aluno: Victor Lira Marques Bastos
e Vinicius Alves Feitosa
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução de Problemas com Grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

O problema

CSES 1682 — Flight Routes Check

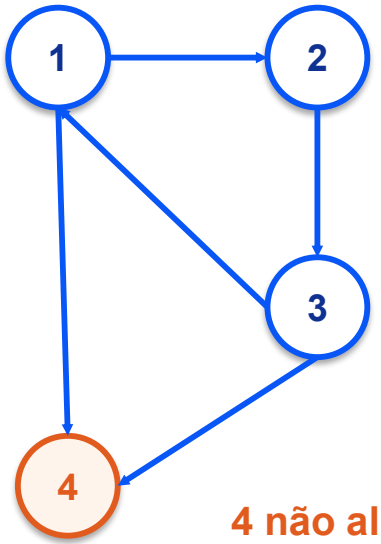
n cidades, m voos dirigidos
Cada voo: a → b

Toda cidade alcança qualquer outra?

YES: todas se alcançam

NO: par a b sem caminho

$n \le 10^5$ $m \le 2 \cdot 10^5$
Solução em tempo linear



Saída

NO
4 1

4 não alcança 1

Conectividade forte

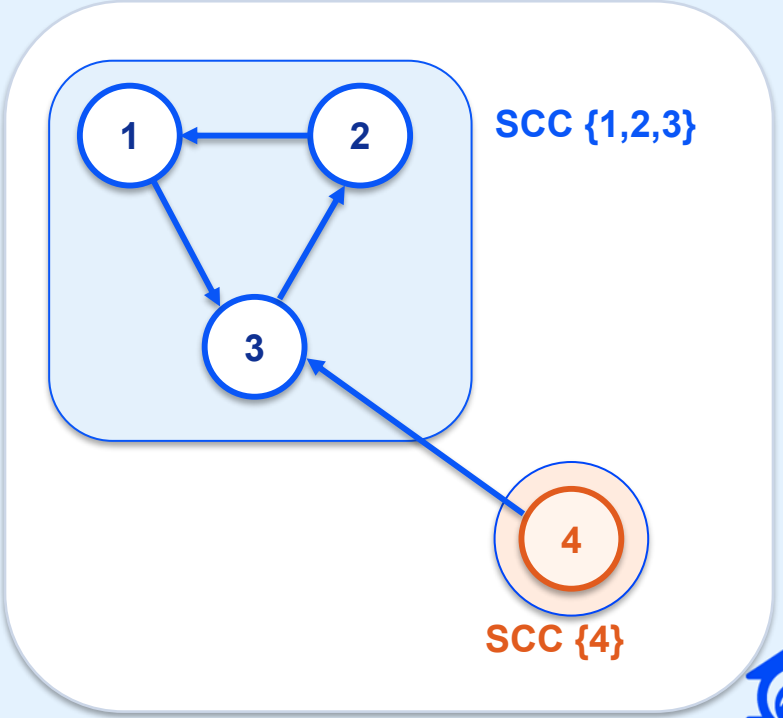
Conectividade forte e componentes fortemente conexas

Para quaisquer u e v, existem caminhos u → v e v → u.

SCC: conjunto máximo de vértices que se alcançam mutuamente.

YES ⇔ uma única SCC

A direção das arestas importa.



Estratégia da solução

Duas DFS e o grafo reverso

1ª DFS

DirectedDFS(g, 0)

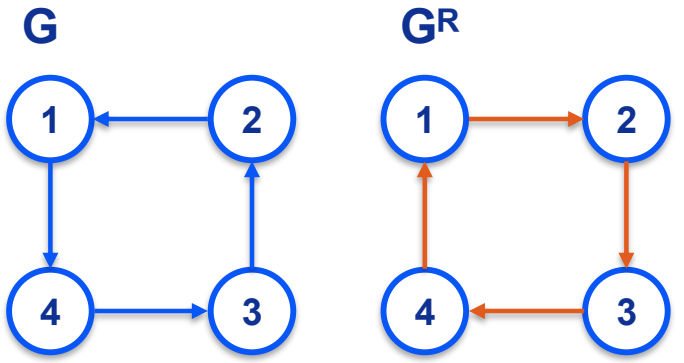
Falha: NO e 1 v

2ª DFS

DirectedDFS(g.reverse(), 0)

Falha: NO e v 1

As duas passam: YES



Cidade 1
índice interno 0

G: v → w **G^R: w → v**

Por que duas DFS são suficientes?

Mesma origem s nas duas buscas

1ª DFS em G

s → todos

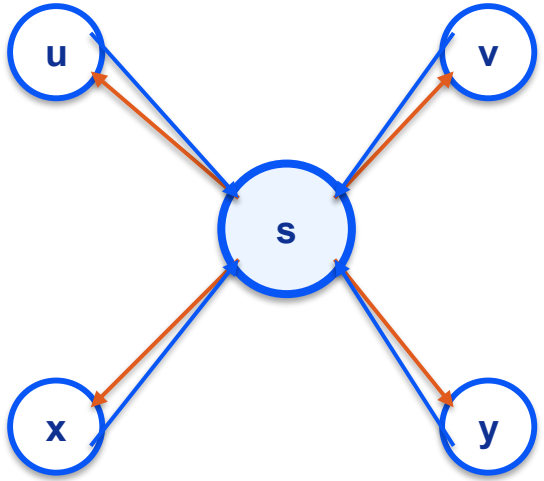
2ª DFS: s alcança todos em G^R

todos → s em G

Logo, para quaisquer u e v:

u → s → v

Assim, G é fortemente conexo.



todos → s e s → todos

Nossa solução × Kosaraju

Mesmos conceitos, objetivos diferentes

Nossa solução

Origem fixa: cidade 1 (índice 0)

DFS em G a partir de 0

DFS em G^R a partir de 0

Ambas alcançam todos?

YES ou NO + testemunho

Verifica se há uma única SCC

Kosaraju

DFS em todo G

Ordem decrescente de término

(pós-ordem reversa)

DFS em G^R nessa ordem

Nova busca por vértice não marcado

Identifica todas as SCCs

Nossa solução usa origem fixa. Não usa o último visitado nem ordem de término.

Implementação final

Representação dirigida e busca sem recursão

Estrutura dos módulos

digraph.py

Lista de adjacência
e reverse()

directed_dfs.py

DFS com pilha explícita

main.py

Duas buscas e testemunhos

Graph → Digraph **Bag → list**

reverse()

```
for v in range(self.V):
    for w in self.adj[v]:
        reversed_graph.add_edge(w, v)
```

DFS iterativa

```
stack = [s]

while stack:
    v = stack.pop()
```

submission.py reúne a solução
Accepted no CSES

Complexidade e validação

Solução linear e resultado no CSES

Custo por etapa

DFS em G	**O(V + E)**
Construção de G^R	**O(V + E)**
DFS em G^R	**O(V + E)**

Tempo total

O(V + E)

Espaço: O(V + E)

Validação

Caso	Saída
1 vértice	YES
ciclo dirigido	YES
falha na 1ª DFS	NO
falha na 2ª DFS	NO
exemplo CSES	NO

CSES: ACCEPTED



Obrigado